

Lab 05: My First REST API — Coffee Menu Service

รายวิชา: CP353002 — Principles of Software Design and Development

ผู้จัดทำ: นายกรกฎ พรหมทอง 673380025-8

นายสรวิษฐ์ ทะมามันท์ 673380295-9

1. รายละเอียดโปรเจกต์

โปรเจกต์นี้เป็น REST API สำหรับจัดการเมนูกาแฟ (Coffee Menu Service) พัฒนาด้วย Spring Boot 3.3.5 และ Java 17 โดยเก็บข้อมูลใน memory (List<Coffee>) แบบไม่ใช้ฐานข้อมูล

โครงสร้างโปรเจกต์ (Layered Design)

com.example.coffeeshop/

|— model/

| |— Coffee.java # เก็บโครงสร้างข้อมูล

|— service/

| |— CoffeeService.java # เก็บ Business Logic + ข้อมูลใน List

|— controller/

| |— CoffeeController.java # รับ-ส่ง HTTP Request/Response

2. ผลการทดสอบ API

2.1 GET /coffees — ดูเมนูทั้งหมด

```
PS C:\Users\korak\OneDrive\Documents\GitHub\Lab05_Spring_Been_Scope\coffeeshop> curl http://localhost:8081/coffees

StatusCode      : 200
StatusDescription : 
Content         : [{"id":1,"name":"Espresso","price":45.0}, {"id":2,"name":"Latte","price":55.0}]
RawContent      : HTTP/1.1 200
                  Transfer-Encoding: chunked
                  Content-Type: application/json
                  Date: Thu, 30 Jul 2026 16:48:44 GMT

                  [{"id":1,"name":"Espresso","price":45.0}, {"id":2,"name":"Latte","price":55.0}]
Forms           : {}
Headers         : [[Transfer-Encoding, chunked], [Content-Type, application/json], [Date, Thu, 30 Jul 2026
                  16:48:44 GMT]]
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 78
```

2.2 GET /coffees/1 — ดูเมนูตาม ID

```
PS C:\Users\korak\OneDrive\Documents\GitHub\Lab05_Spring_Been_Scope\coffeeshop> Invoke-WebRequest -Uri "http://localhost:8081/coffees/1" -Method GET

StatusCode      : 200
StatusDescription : 
Content         : {"id":1,"name":"Espresso","price":45.0}
RawContent      : HTTP/1.1 200
                  Transfer-Encoding: chunked
                  Keep-Alive: timeout=60
                  Connection: keep-alive
                  Content-Type: application/json
                  Date: Thu, 30 Jul 2026 16:50:57 GMT

                  {"id":1,"name":"Espresso","price":45.0...
Forms           : {}
Headers         : [[Transfer-Encoding, chunked], [Keep-Alive, timeout=60], [Connection, keep-alive],
                  [Content-Type, application/json]...]
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 39
```

2.3 POST /coffees — เพิ่มเมนูใหม่

```
PS C:\Users\korak\OneDrive\Documents\GitHub\Lab05_Spring_Been_Scope\coffeeshop> Invoke-WebRequest -Uri "http://localhost:8081/coffees" -Method POST -ContentType "application/json" -Body '{"name":"Cappuccino","price":60.0}'

StatusCode      : 201
StatusDescription : 
Content         : {"id":3,"name":"Cappuccino","price":60.0}
RawContent      : HTTP/1.1 201
                  Transfer-Encoding: chunked
                  Content-Type: application/json
                  Date: Thu, 30 Jul 2026 16:51:48 GMT

                  {"id":3,"name":"Cappuccino","price":60.0}
Forms           : {}
Headers         : [[Transfer-Encoding, chunked], [Content-Type, application/json], [Date, Thu, 30 Jul 2026 16:51:48 GMT]]
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 41
```

2.4 PUT /coffees/2 — แก้ไขเมนู

```
PS C:\Users\korak\OneDrive\Documents\GitHub\Lab05_Spring_Been_Scope\coffeeshop> Invoke-WebRequest -Uri "http://localhost:8081/coffees/2" -Method PUT -ContentType "application/json" -Body '{"name":"Latte","price":50.0}'

StatusCode      : 200
StatusDescription : 
Content         : {"id":2,"name":"Latte","price":50.0}
RawContent      : HTTP/1.1 200
                  Transfer-Encoding: chunked
                  Content-Type: application/json
                  Date: Thu, 30 Jul 2026 16:52:34 GMT

                  {"id":2,"name":"Latte","price":50.0}
Forms           : {}
Headers         : {[Transfer-Encoding, chunked], [Content-Type, application/json], [Date, Thu, 30 Jul 2026 16:52:34 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 36
```

2.5 DELETE /coffees/3 — ลบเมนู

```
PS C:\Users\korak\OneDrive\Documents\GitHub\Lab05_Spring_Been_Scope\coffeeshop> Invoke-WebRequest -Uri "http://localhost:8081/coffees/3" -Method DELETE

StatusCode      : 204
StatusDescription : 
Content         : {}
RawContent      : HTTP/1.1 204
                  Date: Thu, 30 Jul 2026 16:54:01 GMT

Headers         : {[Date, Thu, 30 Jul 2026 16:54:01 GMT]}
RawContentLength : 0
```

2.6 GET /coffees/search?name=Lat — ค้นหาตามชื่อ

```
PS C:\Users\korak\OneDrive\Documents\GitHub\Lab05_Spring_Been_Scope\coffeeshop> Invoke-WebRequest -Uri "http://localhost:8081/coffees/search?name=Lat" -Method GET

StatusCode      : 200
StatusDescription : 
Content         : [{"id":2,"name":"Latte","price":50.0}]
RawContent      : HTTP/1.1 200
                  Transfer-Encoding: chunked
                  Content-Type: application/json
                  Date: Thu, 30 Jul 2026 16:54:34 GMT

                  [{"id":2,"name":"Latte","price":50.0}]
Forms           : {}
Headers         : {[Transfer-Encoding, chunked], [Content-Type, application/json], [Date, Thu, 30 Jul 2026 16:54:34 GMT]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 38
```

2.7 GET /coffees/999 — ทดสอบกรณีไม่พบข้อมูล

```
PS C:\Users\korak\OneDrive\Documents\GitHub\Lab05_Spring_Been_Scope\coffeeshop> Invoke-WebRequest -Uri "http://localhost:8081/coffees/999" -Method GET
Invoke-WebRequest : The remote server returned an error: (404) Not Found.
At line:1 char:1
+ Invoke-WebRequest -Uri "http://localhost:8081/coffees/999" -Method GE ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-WebRequest], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand

PS C:\Users\korak\OneDrive\Documents\GitHub\Lab05_Spring_Been_Scope\coffeeshop>
```

3. Discussion

3.1 HTTP method แต่ละตัว (GET/POST/PUT/DELETE) ต่างกันอย่างไร ยกตัวอย่างจากโปรเจกต์ตัวเอง

ตอบ GET — ใช้สำหรับ อ่าน/ดึงข้อมูล ไม่มีผลกระทบต่อข้อมูลในระบบ

ตัวอย่าง: GET /coffees ใช้ดูเมนูทั้งหมด และ GET /coffees/1 ใช้ดูเมนูเฉพาะรายการ

POST — ใช้สำหรับ สร้างข้อมูลใหม่ จะเพิ่ม resource เข้าไปในระบบ

ตัวอย่าง: POST /coffees ใช้เพิ่มเมนู Cappuccino เข้าไปในใหม่ โดยส่ง JSON ผ่าน Request Body

PUT — ใช้สำหรับ แก้ไขข้อมูลที่มีอยู่ แทนที่ข้อมูลเดิมทั้งหมดด้วยข้อมูลใหม่

ตัวอย่าง: PUT /coffees/2 ใช้แก้ไขราคา Latte จาก 55.0 เป็น 50.0

DELETE — ใช้สำหรับ ลบข้อมูล ออกจากระบบ

ตัวอย่าง: DELETE /coffees/3 ใช้ลบเมนู Cappuccino ออกจาก List

3.2 ทำไมต้องแยก Controller กับ Service ออกจากกัน มีข้อดีอย่างไรถ้าโปรแกรมโตขึ้น

ตอบ การแยก Controller กับ Service เป็นการประยุกต์ใช้หลักการ Separation of Concerns โดย: Controller รับผิดชอบเฉพาะการรับ HTTP Request และส่ง HTTP Response (Presentation Layer) ไม่ต้องสนใจ logic หรือว่าข้อมูลอยู่ที่ไหน

Service รับผิดชอบ Business Logic และการจัดการข้อมูล (Business Layer) ไม่ต้องสนใจว่าจะถูกเรียกผ่าน HTTP หรือช่องทางอื่น

ข้อดีเมื่อโปรแกรมโตขึ้น:

แก้ไขง่าย — ถ้าต้องเปลี่ยนจากเก็บข้อมูลใน List เป็น Database แก้เฉพาะ Service โดยไม่ต้องแตะ Controller

ทดสอบง่าย — สามารถ Unit Test Service ได้โดยไม่ต้องเปิด Web Server

นำกลับมาใช้ใหม่ได้ — Service สามารถถูกเรียกใช้จากหลาย Controller หรือแม้แต่ Background Job ได้

อ่านง่าย — โค้ดเป็นระเบียบ คนที่มาอ่านต่อเข้าใจได้ทันทีว่าส่วนไหนทำอะไร

3.3 ข้อมูลที่เก็บไว้ใน List ใน memory หายไปตอนไหน และถ้าอยากให้ไม่หายควรทำอย่างไร

ตอบ ข้อมูลใน List<Coffee> จะหายไปทันทีเมื่อ:

ปิดหรือ restart Spring Boot application

เกิด OutOfMemoryError หรือ JVM crash

deploy โปรเจกต์ใหม่

แนวทางแก้ไขให้ข้อมูลไม่หาย:

ใช้ฐานข้อมูลจริง เช่น MySQL, PostgreSQL, MongoDB โดยเชื่อมต่อผ่าน Spring Data JPA

ใช้ In-Memory Database เช่น H2 ที่สามารถบันทึกลงไฟล์ได้ (file-based)

เขียนข้อมูลลงไฟล์ เช่น JSON หรือ CSV ก่อนปิดแอป และอ่านกลับมาตอนเริ่มแอป
ใช้ Redis เป็น in-memory data store ที่มี persistence mode เก็บข้อมูลถาวรได้
3.4 @RestController, @GetMapping, @PostMapping, @PathVariable, @RequestBody แต่ละตัวทำหน้าที่อะไร

ตอบ

ตาราง	
Annotation	หน้าที่
@RestController	บอก Spring ว่าคลาสนี้เป็น REST Controller ที่รับ HTTP Request และคืนค่าเป็น JSON/XML โดยอัตโนมัติ รวม @Controller + @ResponseBody ไว้ในตัวเดียว
@GetMapping	ผูก HTTP GET request เข้ากับ method ใช้สำหรับดึงข้อมูล เช่น @GetMapping("/coffees")
@PostMapping	ผูก HTTP POST request เข้ากับ method ใช้สำหรับสร้างข้อมูลใหม่ เช่น @PostMapping("/coffees")
@PathVariable	ดึงค่าจาก URL path เช่น /coffees/{id} โดยนำค่า id มาใส่ตัวแปรใน method
@RequestBody	แปลง JSON จาก HTTP Request Body ให้กลายเป็น Object Java โดยอัตโนมัติ เช่นแปลง {"name":"Latte"} ให้เป็น Coffee object

4. สรุป

โปรเจกต์นี้สามารถสร้าง REST API สำหรับจัดการเมนูกาแฟได้ครบถ้วน แยกชั้น Model/Service/Controller ถูกต้องตามหลักการออกแบบซอฟต์แวร์ รองรับ HTTP Methods ครบทั้ง GET, POST, PUT, DELETE และมีการจัดการกรณีข้อมูลไม่พบด้วย HTTP 404 นอกจากนี้ยังมี endpoint ค้นหาตามชื่อเป็นโบนัสเพิ่มเติม